

REGRESSION PROJECT

June 12, 2026

```
[1]: import pandas as pd
import numpy as np

from datetime import datetime, timedelta

from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

from sklearn.linear_model import LinearRegression, Ridge
from sklearn.ensemble import RandomForestRegressor

import xgboost as xgb

import warnings
warnings.filterwarnings('ignore')
```

```
[2]: df = pd.read_csv(R"C:\Users\Neharika\Downloads\data.csv", encoding='ISO-8859-1')

# Basic cleaning
df = df.dropna(subset=['CustomerID'])
df = df[df['Quantity'] > 0]
df = df[df['UnitPrice'] > 0]

# Convert date
df['InvoiceDate'] = pd.to_datetime(df['InvoiceDate'])

# Revenue column
df['Revenue'] = df['Quantity'] * df['UnitPrice']

# Convert CustomerID to string
df['CustomerID'] = df['CustomerID'].astype(str)

df.head()
```

```
[2]: InvoiceNo StockCode Description Quantity \
0 536365 85123A WHITE HANGING HEART T-LIGHT HOLDER 6
1 536365 71053 WHITE METAL LANTERN 6
2 536365 84406B CREAM CUPID HEARTS COAT HANGER 8
```

3	536365	84029G	KNITTED UNION FLAG HOT WATER BOTTLE	6
4	536365	84029E	RED WOOLLY HOTTIE WHITE HEART.	6

	InvoiceDate	UnitPrice	CustomerID	Country	Revenue
0	2010-12-01 08:26:00	2.55	17850.0	United Kingdom	15.30
1	2010-12-01 08:26:00	3.39	17850.0	United Kingdom	20.34
2	2010-12-01 08:26:00	2.75	17850.0	United Kingdom	22.00
3	2010-12-01 08:26:00	3.39	17850.0	United Kingdom	20.34
4	2010-12-01 08:26:00	3.39	17850.0	United Kingdom	20.34

```
[3]: split_date = df['InvoiceDate'].quantile(0.75)

train_df = df[df['InvoiceDate'] <= split_date]
test_df = df[df['InvoiceDate'] > split_date]

print(train_df.shape, test_df.shape)
```

(298432, 9) (99452, 9)

```
[4]: def create_features(data, reference_date):

    customer_df = data.groupby('CustomerID').agg({
        'InvoiceDate': ['max', 'min'],
        'InvoiceNo': 'nunique',
        'Revenue': ['sum', 'mean'],
        'StockCode': 'nunique'
    }).reset_index()

    customer_df.columns = [
        'CustomerID',
        'last_purchase',
        'first_purchase',
        'frequency',
        'monetary_total',
        'avg_order_value',
        'product_diversity'
    ]

    # Recency
    customer_df['recency'] = (reference_date - customer_df['last_purchase']).dt.
    ↪days

    # Tenure
    customer_df['tenure'] = (customer_df['last_purchase'] -
    ↪customer_df['first_purchase']).dt.days

    # Purchase velocity
```

```

customer_df['purchase_velocity'] = customer_df['frequency'] /
↳ (customer_df['tenure'] + 1)

return customer_df

```

```

[5]: reference_date_train = train_df['InvoiceDate'].max()

train_features = create_features(train_df, reference_date_train)

```

```

[6]: future_revenue = test_df.groupby('CustomerID')['Revenue'].sum().reset_index()
future_revenue.columns = ['CustomerID', 'CLV']

# Merge with features
model_df = train_features.merge(future_revenue, on='CustomerID', how='left')

# Fill missing CLV (no future purchase)
model_df['CLV'] = model_df['CLV'].fillna(0)

model_df.head()

```

```

[6]: CustomerID      last_purchase      first_purchase  frequency \
0      12346.0  2011-01-18 10:01:00  2011-01-18 10:01:00         1
1      12347.0  2011-08-02 08:48:00  2010-12-07 14:57:00         5
2      12348.0  2011-09-25 13:13:00  2010-12-16 19:09:00         4
3      12350.0  2011-02-02 16:01:00  2011-02-02 16:01:00         1
4      12352.0  2011-09-28 14:58:00  2011-02-16 12:33:00         7

      monetary_total  avg_order_value  product_diversity  recency  tenure \
0          77183.60      77183.600000                 1      275         0
1          2790.86      22.506935                 82       79      237
2          1797.24      57.975484                 22       25      282
3           334.40      19.670588                 17     259         0
4          2194.31      31.347286                 47       21     224

      purchase_velocity      CLV
0          1.000000      0.00
1          0.021008  1519.14
2          0.014134      0.00
3          1.000000      0.00
4          0.031111   311.73

```

```

[7]: features = [
      'recency',
      'frequency',
      'monetary_total',
      'avg_order_value',
      'product_diversity',

```

```

        'tenure',
        'purchase_velocity'
    ]

    X = model_df[features]
    y = model_df['CLV']

    # Train-test split (within training set for validation)
    X_train, X_val, y_train, y_val = train_test_split(X, y, test_size=0.2,
        ↪random_state=42)

```

```

[8]: lr = LinearRegression()
     lr.fit(X_train, y_train)

     lr_pred = lr.predict(X_val)

```

```

[9]: ridge = Ridge(alpha=1.0)
     ridge.fit(X_train, y_train)

     ridge_pred = ridge.predict(X_val)

```

```

[11]: rf = RandomForestRegressor(n_estimators=100, random_state=42)
     rf.fit(X_train, y_train)

     rf_pred = rf.predict(X_val)

```

```

[12]: xgb_model = xgb.XGBRegressor(
        n_estimators=200,
        max_depth=5,
        learning_rate=0.05,
        random_state=42
    )

     xgb_model.fit(X_train, y_train)

     xgb_pred = xgb_model.predict(X_val)

```

```

[13]: def evaluate_model(y_true, y_pred, model_name):
        rmse = np.sqrt(mean_squared_error(y_true, y_pred))
        mae = mean_absolute_error(y_true, y_pred)
        r2 = r2_score(y_true, y_pred)

        print(f"{model_name}")
        print(f"RMSE: {rmse:.2f}")
        print(f"MAE : {mae:.2f}")
        print(f"R2  : {r2:.2f}")
        print("-"*30)

```

```
[14]: evaluate_model(y_val, lr_pred, "Linear Regression")
      evaluate_model(y_val, ridge_pred, "Ridge")
      evaluate_model(y_val, rf_pred, "Random Forest")
      evaluate_model(y_val, xgb_pred, "XGBoost")
```

Linear Regression

RMSE: 1943.14

MAE : 466.54

R2 : 0.25

Ridge

RMSE: 1943.13

MAE : 466.54

R2 : 0.25

Random Forest

RMSE: 1784.69

MAE : 463.35

R2 : 0.37

XGBoost

RMSE: 6301.61

MAE : 676.92

R2 : -6.87

```
[15]: final_model = xgb_model

      model_df['predicted_clv'] = final_model.predict(X)
```

```
[16]: model_df['segment'] = pd.qcut(
      model_df['predicted_clv'],
      q=3,
      labels=['Low Value', 'Medium Value', 'High Value']
      )

      model_df.head()
```

```
[16]: CustomerID      last_purchase      first_purchase  frequency  \
0      12346.0  2011-01-18 10:01:00  2011-01-18 10:01:00      1
1      12347.0  2011-08-02 08:48:00  2010-12-07 14:57:00      5
2      12348.0  2011-09-25 13:13:00  2010-12-16 19:09:00      4
3      12350.0  2011-02-02 16:01:00  2011-02-02 16:01:00      1
4      12352.0  2011-09-28 14:58:00  2011-02-16 12:33:00      7

      monetary_total  avg_order_value  product_diversity  recency  tenure  \
0      77183.60      77183.600000      1      275      0
1      2790.86      22.506935      82      79      237
```

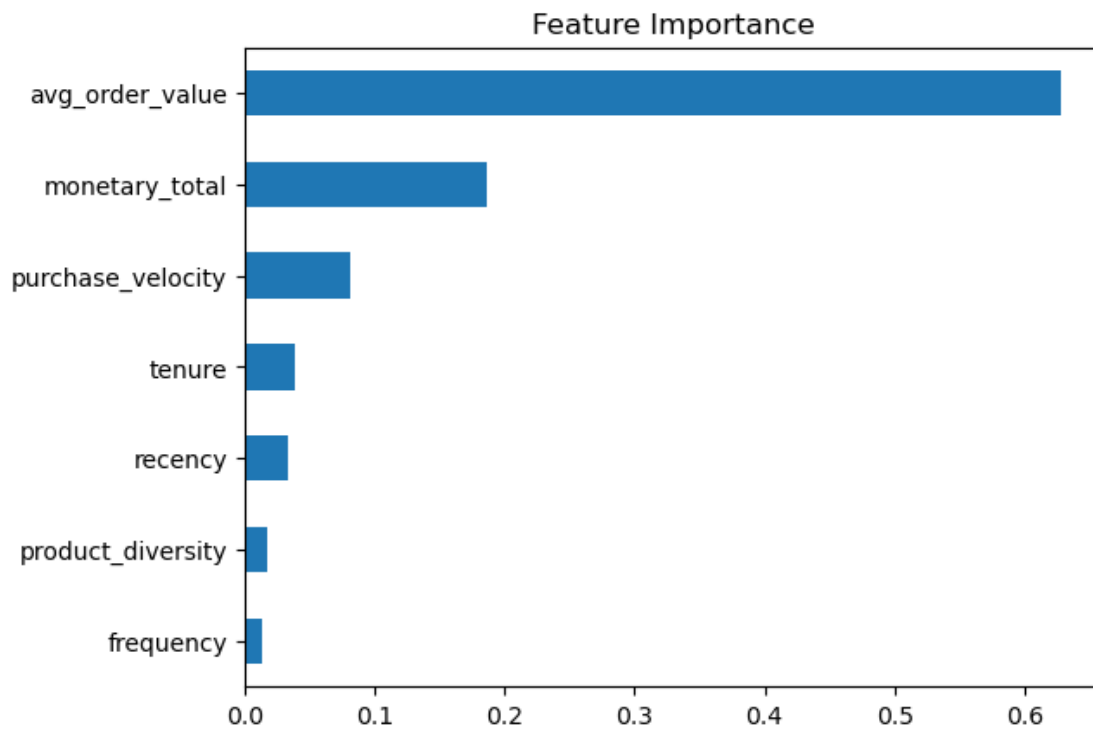
2	1797.24	57.975484	22	25	282
3	334.40	19.670588	17	259	0
4	2194.31	31.347286	47	21	224

	purchase_velocity	CLV	predicted_clv	segment
0	1.000000	0.00	22394.535156	High Value
1	0.021008	1519.14	585.388977	High Value
2	0.014134	0.00	426.920502	High Value
3	1.000000	0.00	66.393471	Low Value
4	0.031111	311.73	481.869415	High Value

```
[17]: import matplotlib.pyplot as plt
```

```
importance = pd.Series(final_model.feature_importances_, index=features)
importance.sort_values().plot(kind='barh')
```

```
plt.title("Feature Importance")
plt.show()
```



```
[18]: top_20 = model_df.nlargest(int(0.2 * len(model_df)), 'predicted_clv')
```

```
revenue_share = top_20['predicted_clv'].sum() / model_df['predicted_clv'].sum()
```

```
print(f"Top 20% customers contribute {revenue_share:.2%} of predicted revenue")
```

Top 20% customers contribute 75.63% of predicted revenue

```
[19]: # Simple churn proxy
model_df['churn_risk'] = np.where(model_df['recency'] > 90, 1, 0)

model_df['expected_value'] = model_df['predicted_clv'] * (1 -
↳model_df['churn_risk'])
```

```
[20]: pip install shap
```

```
Requirement already satisfied: shap in c:\users\neharika\anaconda3\lib\site-
packages (0.48.0)
Requirement already satisfied: numpy in c:\users\neharika\anaconda3\lib\site-
packages (from shap) (2.1.3)
Requirement already satisfied: scipy in c:\users\neharika\anaconda3\lib\site-
packages (from shap) (1.15.3)
Requirement already satisfied: scikit-learn in
c:\users\neharika\anaconda3\lib\site-packages (from shap) (1.6.1)
Requirement already satisfied: pandas in c:\users\neharika\anaconda3\lib\site-
packages (from shap) (2.2.3)
Requirement already satisfied: tqdm>=4.27.0 in
c:\users\neharika\anaconda3\lib\site-packages (from shap) (4.67.1)
Requirement already satisfied: packaging>20.9 in
c:\users\neharika\appdata\roaming\python\python313\site-packages (from shap)
(25.0)
Requirement already satisfied: slicer==0.0.8 in
c:\users\neharika\anaconda3\lib\site-packages (from shap) (0.0.8)
Requirement already satisfied: numba>=0.54 in
c:\users\neharika\anaconda3\lib\site-packages (from shap) (0.61.0)
Requirement already satisfied: cloudpickle in
c:\users\neharika\anaconda3\lib\site-packages (from shap) (3.0.0)
Requirement already satisfied: typing-extensions in
c:\users\neharika\appdata\roaming\python\python313\site-packages (from shap)
(4.14.0)
Requirement already satisfied: llvmlite<0.45,>=0.44.0dev0 in
c:\users\neharika\anaconda3\lib\site-packages (from numba>=0.54->shap) (0.44.0)
Requirement already satisfied: colorama in
c:\users\neharika\appdata\roaming\python\python313\site-packages (from
tqdm>=4.27.0->shap) (0.4.6)
Requirement already satisfied: python-dateutil>=2.8.2 in
c:\users\neharika\appdata\roaming\python\python313\site-packages (from
pandas->shap) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in
c:\users\neharika\anaconda3\lib\site-packages (from pandas->shap) (2024.1)
Requirement already satisfied: tzdata>=2022.7 in
c:\users\neharika\anaconda3\lib\site-packages (from pandas->shap) (2025.2)
```

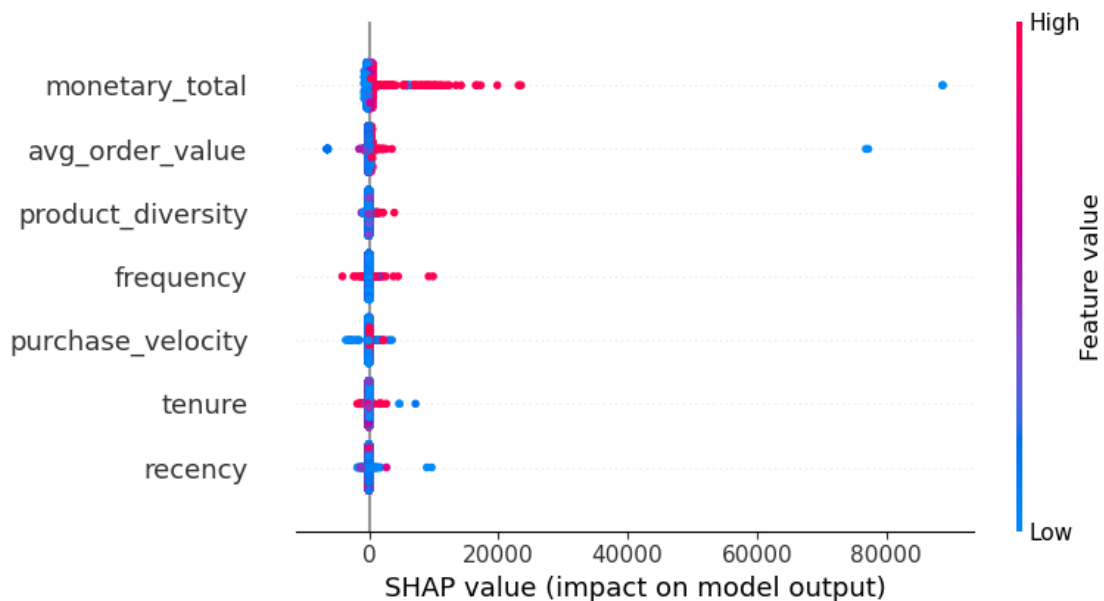
Requirement already satisfied: six>=1.5 in
c:\users\neharika\appdata\roaming\python\python313\site-packages (from python-dateutil>=2.8.2->pandas->shap) (1.17.0)
Requirement already satisfied: joblib>=1.2.0 in
c:\users\neharika\anaconda3\lib\site-packages (from scikit-learn->shap) (1.4.2)
Requirement already satisfied: threadpoolctl>=3.1.0 in
c:\users\neharika\anaconda3\lib\site-packages (from scikit-learn->shap) (3.5.0)
Note: you may need to restart the kernel to use updated packages.

```
[21]: import shap

# Create explainer
explainer = shap.TreeExplainer(final_model)

# Compute SHAP values
shap_values = explainer.shap_values(X)

# Summary plot (global importance)
shap.summary_plot(shap_values, X)
```



```
[23]: # Simulate retention improvement
improved_clv = model_df['predicted_clv'] * 1.1

lift = improved_clv.sum() - model_df['predicted_clv'].sum()

print(f"Revenue increase if retention improves by 10%: {lift:.2f}")
```

Revenue increase if retention improves by 10%: 198636.50

[]: